Лабораторна робота №9. Створення інтерактивного DOM та обробка подій: Event Delegation, форми та доступність динамічного інтерфейсу

Термін здачі: 19.10.2026

1. Мета роботи

Навчитися будувати інтерактивний інтерфейс на DOM API без фреймворків: створювати елементи з шаблону, обробляти події через делегування, працювати з формами й модальним діалогом і керувати фокусом так, щоб інтерфейсом можна було користуватися з клавіатури та скрінрідера. Навчитися перевіряти JavaScript-код, написаний AI-агентом, за специфікацією й поведінкою в браузері, а не за тим, що він «начебто працює».

Очікуваний результат: застосунок «Мій список», кожну частину якого спершу написав AI-агент, а ви знайшли розбіжності з ТЗ і виправили їх власноруч; «Пройдено: 33 з 33» у самоперевірці й історія Git, де за комітом кожної AI-чернетки йде коміт ваших виправлень.

2. Цілі та результати навчання

Після виконання лабораторної роботи студент повинен:

- Знаходити елементи через `querySelector*`, розрізняти live і static колекції; пам'ятати, що значення `dataset` — рядки.
- Створювати елементи з `<template>`, записувати дані через `textContent` і атрибути; пояснювати, чому `innerHTML` з даними — це XSS.
- Пояснювати фази capturing і bubbling, різницю між `target` і `currentTarget`, значення `this` в обробнику; використовувати `closest()`.
- Застосовувати event delegation і пояснювати, чому обробник прив'язаний до вузла, а не до селектора.

- Обробляти форми: `submit` і `preventDefault()`, надсилання клавішею Enter, `input` і `change`, `reset`, `valueAsNumber`, Constraint Validation API.
- Керувати фокусом: `focus()`, `tabindex`, фокус після видалення елемента, модальний `<dialog>`.
- Передавати стан допоміжним технологіям: `aria-pressed`, `aria-invalid`, `aria-describedby`, live region (`role="status"`).
- Перевіряти AI-згенерований код за ТЗ, самоперевіркою й DevTools і фіксувати аудит у Git.

3. Завдання лабораторної роботи

1. **Удома, до заняття:** опрацювати дослідницькі завдання (§4) і провести експерименти: для кожного **до** запуску проаналізувати код і записати очікуваний результат.
2. **На занятті:** завантажити starter-проєкт «Мій список», зафіксувати його в Git без змін і переконатися, що самоперевірка показує «Пройдено: 0 з 33» (§5.1).
3. **На занятті:** виконати Challenges 1–4 (§5.3–5.6): для кожного AI-агент пише чернетку, ви фіксуєте її, перевіряєте за ТЗ і виправляєте власноруч.
4. **Удома:** пройти ручний чек-лист ТЗ на фінальній версії, опублікувати проєкт на GitHub Pages або Vercel, оформити звіт (§6). Потім — захист (§7).

4. Дослідницькі завдання

Перед практичною частиною опрацюйте джерела й проведіть експерименти. Загальна довідка — Introduction to the DOM — MDN; вимоги доступності — WCAG 2.2 і ARIA Authoring Practices Guide; поведінку браузера визначає HTML Living Standard.

Експерименти досліджують поведінку браузера, а не код застосунку: у них нічого не треба виправляти. Рядок «Де це трапиться» в кожному експерименті показує, у якій частині застосунку «Мій список» ця поведінка стане помилкою, якщо її не врахувати.

Сторінка для експериментів. У starter-проєкті (watchlist-starter.zip, той самий, що в §5) є сторінка `experiments.html`. На ній 15 пронумерованих блоків — по одному на експеримент. Номер і назва блоку збігаються з номером і назвою експерименту нижче. Кожен блок — елемент `<section id="exp-N">`, і фрагмент коду знаходить свої елементи саме в ньому: `document.querySelector('#exp-1 …')`. Опис кожного експерименту показує HTML його блоку, тож код можна проаналізувати, не відкриваючи сторінки.

Розпакуйте архів і запустіть Live Preview для `experiments.html`. Сторінку відкрийте в браузері за адресою `http://127.0.0.1:…`, яку показує Live Preview, а не у вбудованій вкладці VS Code, і перейдіть у DevTools (F12, на macOS — Cmd+Option+I) → Console.

Потрібен саме Chrome або Edge: Live Expression і Preserve log є в їхніх DevTools, а клік і Tab у Safari та Firefox на macOS ставлять фокус інакше (ТЗ, §6).

Протокол експерименту:

1. Прочитайте опис, **проаналізуйте** код і запишіть очікуваний результат — відповідь на питання «Проаналізуйте».
2. Оновіть сторінку `experiments.html` і вставте фрагмент у Console цілим. Сторінку оновлюйте перед кожним експериментом: фрагменти змінюють DOM.
3. Якщо хоч один очікуваний результат не збігся з фактичним, поясніть, де саме аналіз був хибним.

Питання в рядку «Де це трапиться» — для підготовки до захисту, письмово на них відповідати не треба. Результати зведіть у таблицю — вона входить у звіт:

#	Що досліджено	Мій аналіз	Фактичний результат	Пояснення (якщо не збіглося)
1	Live і static колекції	...	...	...

4.1. DOM API

Опрацювати:

- Locating DOM elements using selectors — MDN
- HTMLCollection і NodeList — MDN
- Node: textContent і Element: innerHTML, розділ Security considerations — MDN
- <template>, Element: replaceChildren(), HTMLElement: dataset — MDN

Дослідити: що повертають `querySelector` і `querySelectorAll`, якщо нічого не знайдено; live і static колекції; `textContent`, `innerHTML` і `innerText`; чому вміст `<template>` інертний; що робить `cloneNode(true)`.

Експерименти:

1. **Live і static колекції.** Блок `#exp-1` на сторінці `experiments.html`:

```
<section id="exp-1">
  <h2>1 · Live і static колекції</h2>
  <ul>
    <li class="item">Перший</li>
    <li class="item">Другий</li>
    <li class="item">Третій</li>
    <li class="item">Четвертий</li>
    <li class="item">П'ятий</li>
    <li class="item">Шостий</li>
  </ul>
</section>
```

Фрагмент бере ці пункти через `getElementsByClassName` і в циклі прибирає в кожного клас `item`. Останній рядок рахує, скільки елементів із цим класом лишилося.

```
const items = document.getElementsByClassName('item');
for (let i = 0; i < items.length; i++) items[i].classList.remove('item');
document.querySelectorAll('.item').length;
```

Проаналізуйте: яке число поверне останній рядок? Потім оновіть сторінку, замініть перший рядок на `const items = document.querySelectorAll('.item');` і проаналізуйте ще раз.
Де це трапиться: очищення списку `#results` перед новим показом карток (Challenge 1): цикл, що видаляє картки по одній, обходить live-колекцію `list.children`. Поясніть, чим відрізняються ці дві колекції, яку з них безпечно обходити, коли цикл змінює DOM, і що замість такого циклу робить `replaceChildren()`.

2. **Рядок із HTML чотирма способами.** Блок `#exp-2` на сторінці `experiments.html`:

```
<section id="exp-2">
  <h2>2 · Рядок із HTML чотирма способами</h2>
  <p>textContent: <output class="as-text"></output></p>
  <p>innerHTML: <output class="as-html"></output></p>
</section>
```

`payload` створює рядок із тегом `<img>`: зображення не існує, а обробник `onerror` виводить у Console, яким способом рядок потрапив на сторінку, — але лише якщо браузер створив із рядка елемент `<img>` і почав завантажувати зображення. Фрагмент записує рядок через `textContent` у перший `<output>`, через `innerHTML` — у другий, а також через `innerHTML` тимчасового `<div>`, якого немає в документі, і через `innerHTML` елемента `<template>`.

```
const payload = (via) => `<img src="x" onerror="console.log('onerror:
${via}')">`;
document.querySelector('#exp-2 .as-text').textContent =
payload('textContent');
document.querySelector('#exp-2 .as-html').innerHTML = payload('innerHTML');
document.createElement('div').innerHTML = payload('div');
document.createElement('template').innerHTML = payload('template');
```

Проаналізуйте: які з чотирьох способів виведуть повідомлення в Console? Що буде видно в розділі 2 сторінки?
Де це трапиться: назва серіалу або нотатка, у якій є розмітка (Challenge 1 і 4), і опис серіалу, що в даних TVmaze сам є HTML (Challenge 4). Поясніть, які способи безпечні для даних і чому «очистити» HTML від тегів через тимчасовий `<div>` не можна.

3. **Значення з dataset.** Блок `#exp-3` на сторінці `experiments.html`:

```
<section id="exp-3">
  <h2>3 · Значення з dataset</h2>
  <ul>
    <li data-id="82">Game of Thrones</li>
  </ul>
</section>
```

   Так само кожна картка застосунку зберігатиме id свого серіалу. Фрагмент читає `data-id` і порівнює його з числом 82.

```
const li = document.querySelector('#exp-3 li');
[li.dataset.id, typeof li.dataset.id, li.dataset.id === 82,
Number(li.dataset.id) === 82];
```

   Проаналізуйте: які чотири значення буде в масиві?
   Де це трапиться: id, прочитаний з картки, передають у функції `src/state.js` (Challenge 2). Поясніть, чому порівняння з числом не спрацьовує і як отримати число.

4. **`innerHTML +=` і обробники.** Блок `#exp-4` на сторінці `experiments.html`:

```
<section id="exp-4">
  <h2>4 · innerHTML += і обробники</h2>
  <button type="button">Кнопка з обробником</button>
</section>
```

   Фрагмент додає кнопці обробник `click` і клікає по ній, потім дописує в блок рядок через `innerHTML +=` і клікає по кнопці знову.

```
const box = document.querySelector('#exp-4');
box.querySelector('button').addEventListener('click', () =>
console.log('клік'));
box.querySelector('button').click();
box.innerHTML += '<p>Новий рядок</p>';
box.querySelector('button').click();
```

   Проаналізуйте: скільки разів Console виведе «клік»?
   Де це трапиться: список, до якого нові картки «дописують» рядком HTML (Challenge 1 і 2). Поясніть, що `innerHTML +=` робить з елементами, які вже є в контейнері.

4.2. Події та event delegation

Опрацювати:

- Introduction to events і Event bubbling, зокрема розділ Event delegation — MDN

- EventTarget: addEventListener() — MDN, зокрема розділи про `this` і про повторну реєстрацію
- Event: target, Event: currentTarget, Element: closest() — MDN
- Button pattern — ARIA APG

Дослідити: три фази поширення події; `target` і `currentTarget`; чому `this` у function-обробнику — елемент, а в arrow function — ні; що повертає `closest()`, якщо предка немає; що буде, якщо додати обробник двічі; чому делегований обробник працює для елементів, створених пізніше; що `<button>` уміє порівняно з `<div>` з обробником `click`; що означає `isTrusted`.

Експерименти:

5. **Фази поширення події.** Блок `#exp-5` на сторінці `experiments.html`:

   ```
   <section id="exp-5">
     <h2>5 · Фази поширення події</h2>
     <ul>
       <li><button type="button">Клікніть мене</button></li>
     </ul>
   </section>
   ```

   Фрагмент додає `<ul>`, `<li>` і кнопці по два обробники `click`: один для фази capturing, другий — для bubbling. Після запуску клікніть кнопку мишею.

   ```
   const ul = document.querySelector('#exp-5 ul');
   for (const el of [ul, ul.querySelector('li'), ul.querySelector('button')])
   {
     const name = el.tagName.toLowerCase();
     el.addEventListener('click', () => console.log(name, 'capture'), {
   capture: true });
     el.addEventListener('click', () => console.log(name, 'bubble'));
   }
   ```

Проаналізуйте: у якому порядку з'являться шість рядків?
Де це трапиться: обробник на всьому списку карток (Challenge 2) отримує клік по кнопці саме завдяки цим фазам. Поясніть порядок через три фази.

6. **`target`, `currentTarget` і `this`.** Блок `#exp-6` на сторінці `experiments.html`:

```
<section id="exp-6">
  <h2>6 · target, currentTarget і this</h2>
  <button type="button">
    <svg viewBox="0 0 24 24" aria-hidden="true" focusable="false">
      <path d="…" fill="currentColor" />
    </svg>
    Кнопка з іконкою
  </button>
</section>
```

`<svg>` — іконка-зірка, `<path>` — її контур. Фрагмент додає кнопці два обробники — звичайною функцією й arrow function — і ще один обробник на весь блок. Після запуску клікніть мишею по зафарбованій частині зірки, потім по тексту кнопки, потім по заголовку блоку.

```
const button = document.querySelector('#exp-6 button');
button.addEventListener('click', function (event) {
  console.log('function:', event.target.tagName,
event.currentTarget.tagName, this.tagName);
});
button.addEventListener('click', (event) => {
  console.log('arrow:', event.target.tagName, event.currentTarget ===
button, this === window);
});
document.querySelector('#exp-6').addEventListener('click', (event) => {
  console.log('closest:', event.target.closest('button'));
});
```

Проаналізуйте: що виведе кожен рядок для кожного з трьох кліків?
Де це трапиться: кнопка «До списку» на картці теж має іконку (Challenge 2). Поясніть, від чого залежить `event.target` і що повертає `closest()`, коли кнопки серед предків немає.

7. **Той самий обробник двічі.** Блок `#exp-7` на сторінці `experiments.html`:

```
<section id="exp-7">
  <h2>7 · Той самий обробник двічі</h2>
  <button type="button">Лічильник</button>
</section>
```

Фрагмент реєструє на кнопці чотири обробники, що збільшують лічильник `count`: дві однакові на вигляд arrow functions і двічі ту саму функцію `named`. Потім один раз клікає по кнопці.

```
const button = document.querySelector('#exp-7 button');
let count = 0;
const named = () => count++;
button.addEventListener('click', () => count++);
button.addEventListener('click', () => count++);
button.addEventListener('click', named);
button.addEventListener('click', named);
button.click();
count;
```

Проаналізуйте: чому дорівнюватиме `count`?
Де це трапиться: обробник, який додають у функції, що перемальовує список (Challenge 2). Поясніть, коли браузер реєструє обробник повторно, а коли ні.

8. **`<div>` чи `<button>`.** Блок `#exp-8` на сторінці `experiments.html`:

```
<section id="exp-8">
  <h2>8 · div чи button</h2>
  <div class="fake">Div з обробником</div>
  <button type="button">Справжня кнопка</button>
</section>
```

Клас `fake` лише оформлює `<div>` як кнопку. Фрагмент додає обом елементам обробник `click`, що виводить значення `isTrusted`. Після запуску:

- клікніть обидва елементи мишею;
- клікніть кнопку «Лічильник» у блоці `#exp-7` і натискайте Tab;
- коли фокус на кнопці «Справжня кнопка», натисніть Enter, потім Space;
- виконайте в Console `div.click(); button.click();`.

```
const div = document.querySelector('#exp-8 .fake');
const button = document.querySelector('#exp-8 button');
div.addEventListener('click', (event) => console.log('div',
event.isTrusted));
button.addEventListener('click', (event) => console.log('button',
event.isTrusted));
```

Проаналізуйте: що виведе Console після кожної дії? Чи отримає `<div>` фокус клавішею Tab?
Де це трапиться: кнопки на картках і в діалозі (Challenge 1, 2 і 4). Поясніть, що `<button>` уміє без JavaScript і що означає `isTrusted`.

4.3. Форми

Опрацювати:

- Client-side form validation — MDN
- Constraint validation API і HTMLInputElement: setCustomValidity() — MDN
- HTMLFormElement: submit event, reset event, requestSubmit() — MDN
- Element: input event і change event — MDN
- Implicit submission — HTML Living Standard

Дослідити: коли форму надсилає Enter і що станеться без `preventDefault()`; чим `submit` відрізняється від `click` по кнопці; коли настають `input` і `change`; коли настає `reset` відносно очищення полів; чим `value` відрізняється від `valueAsNumber`; що таке `validity`, `checkValidity()` і `setCustomValidity()`; що робить атрибут `novalidate`.

Експерименти:

9. **Enter у формі.** Блок `#exp-9` на сторінці `experiments.html`:

   ```
   <section id="exp-9">
     <h2>9 · Enter у формі</h2>
     <form>
       <label>Пошук <input name="q" autocomplete="off" /></label>
       <button>Знайти</button>
     </form>
   </section>
   ```

   Фрагмент додає кнопці обробник `click`, а формі — обробник `submit`. Спершу увімкніть у Console **Preserve log**, інакше записи зникнуть разом зі сторінкою. Після запуску клікніть у поле, введіть `drama` і натисніть Enter.

   ```
   const form = document.querySelector('#exp-9 form');
   form.querySelector('button').addEventListener('click', (event) =>
   console.log('click', event.isTrusted));
   form.addEventListener('submit', () => console.log('submit'));
   ```

   Проаналізуйте: які записи з'являться в Console? Що станеться зі сторінкою й адресою в рядку браузера?
   Де це трапиться: поле пошуку у формі фільтрів (Challenge 3) і форма нотатки (Challenge 4). Поясніть, що запускає надсилання форми і як його скасувати.

10. **`value` і `valueAsNumber`.** Блок `#exp-10` на сторінці `experiments.html`:

    ```
    <section id="exp-10">
      <h2>10 · value і valueAsNumber</h2>
      <label>Число <input type="number" /></label>
    </section>
    ```

Фрагмент записує в поле 7 і додає до значення 1 двома способами, потім очищує поле й повторює.

```
const input = document.querySelector('#exp-10 input');
input.value = '7';
const filled = [input.value + 1, input.valueAsNumber + 1];
input.value = '';
[filled, [input.value + 1, input.valueAsNumber + 1]];
```

Проаналізуйте: які дві пари значень поверне фрагмент?
Де це трапиться: поле «Моя оцінка» (Challenge 4), адже `saveNote()` приймає оцінку лише числом. Поясніть, якого типу кожне значення і чому для порожнього поля одне з них — `NaN`.

11. **`setCustomValidity`.** Блок `#exp-11` на сторінці `experiments.html`:

```
<section id="exp-11">
  <h2>11 · setCustomValidity</h2>
  <label>Оцінка від 1 до 10 <input type="number" min="1" max="10" />
</label>
</section>
```

Фрагмент вводить 42, позначає поле як невалідне власним повідомленням, потім виправляє значення на 5 і перевіряє валідність.

```
const input = document.querySelector('#exp-11 input');
input.value = '42';
if (input.valueAsNumber > 10) input.setCustomValidity('Не більше 10');
input.value = '5';
[input.validity.valid, input.validity.customError, input.checkValidity(),
input.validationMessage];
```

Проаналізуйте: які чотири значення буде в масиві?
Де це трапиться: перевірка оцінки у формі нотатки (Challenge 4). Поясніть, коли повідомлення, задане через `setCustomValidity`, зникає.

12. **Подія `reset`.** Блок `#exp-12` на сторінці `experiments.html`:

```
<section id="exp-12">
  <h2>12 · Подія reset</h2>
  <form>
    <label>Фільтр <input name="q" /></label>
    <button type="reset">Скинути</button>
  </form>
</section>
```

Фрагмент додає формі обробник `reset`, що виводить значення поля, записує в поле `drama` і скидає форму методом `reset()`.

```
const form = document.querySelector('#exp-12 form');
form.addEventListener('reset', () => console.log('y reset:',
JSON.stringify(form.elements.q.value)));
form.elements.q.value = 'drama';
form.reset();
console.log('після reset():', JSON.stringify(form.elements.q.value));
```

Проаналізуйте: які два рядки виведе Console? Потім введіть у поле щось інше й натисніть «Скинути» мишею: що виведе обробник цього разу?
Де це трапиться: кнопка «Скинути» у формі фільтрів (Challenge 3). Поясніть, коли настає подія `reset` відносно очищення полів.

4.4. Фокус, діалоги й доступність

Опрацювати:

- HTMLElement: focus() і tabindex — MDN
- <dialog> і HTMLElement: inert — MDN; Dialog (Modal) pattern — ARIA APG
- ARIA live regions, aria-pressed, aria-invalid, aria-describedby — MDN
- WCAG 2.2: 2.1.1 Keyboard, 2.4.3 Focus Order, 2.4.7 Focus Visible, 2.5.3 Label in Name, 3.3.1 Error Identification, 4.1.2 Name, Role, Value, 4.1.3 Status Messages

Дослідити: які елементи отримують фокус без `tabindex`; `tabindex="-1"` і `tabindex="0"`; куди переходить фокус, коли елемент із фокусом видалено; що робить `showModal()` з рештою сторінки і куди повертає фокус `close()`; що таке accessible name і звідки його бере браузер; чому live region має бути в документі до появи тексту. Почути оголошення можна лише скрінрідером (Narrator, NVDA, VoiceOver) — це необов'язково.

Для експериментів 13–15 додайте в Console **Live Expression** `document.activeElement`: воно постійно показує елемент, на якому зараз фокус.

Експерименти:

13. **Видалення елемента з фокусом.** Блок `#exp-13` на сторінці `experiments.html`:

```
<section id="exp-13">
  <h2>13 · Видалення елемента з фокусом</h2>
  <button type="button">Мене видалять</button>
</section>
```

Фрагмент ставить на кнопку фокус, видаляє її з документа й читає, де тепер фокус.

```
const button = document.querySelector('#exp-13 button');
button.focus();
const before = document.activeElement === button;
button.remove();
[before, document.activeElement];
```

Проаналізуйте: які два значення буде в масиві?
Де це трапиться: список перемальовано, а фокус був на кнопці старої картки (Challenge 2, 3 і 4). Поясніть, куди переходить фокус і що це означає для користувача клавіатури.

14. **`focus()` і `tabindex`.** Блок `#exp-14` на сторінці `experiments.html`:

```
<section id="exp-14">
  <h2>14 · focus() і tabindex</h2>
  <div>Звичайний div</div>
</section>
```

Фрагмент викликає на `<div>` метод `focus()`, потім додає йому `tabindex="-1"` і викликає `focus()` ще раз.

```
const box = document.querySelector('#exp-14 div');
box.focus();
const plain = document.activeElement === box;
box.tabIndex = -1;
box.focus();
[plain, document.activeElement === box];
```

Проаналізуйте: які два значення буде в масиві? Потім клікніть кнопку в блоці `#exp-13` і натисніть Tab: чи потрапить фокус на цей `<div>`?
Де це трапиться: коли в списку не лишилося карток, фокус треба поставити на заголовок списку (Challenge 3). Поясніть, чим `tabindex="-1"` відрізняється від `tabindex="0"`.

15. **Модальний діалог і фокус.** Блок `#exp-15` на сторінці `experiments.html`:

```
<section id="exp-15">
  <h2>15 · Модальний діалог і фокус</h2>
  <button type="button" class="opener">Відкрити діалог</button>
  <button type="button" class="outside">Кнопка поза діалогом</button>
  <dialog>
    <p>Модальний діалог</p>
    <button type="button">Кнопка в діалозі</button>
  </dialog>
</section>
```

Фрагмент ставить фокус на «Відкрити діалог», відкриває діалог через `showModal()`, пробує перевести фокус на кнопку поза діалогом, а потім закриває діалог.

```
const section = document.querySelector('#exp-15');
const dialog = section.querySelector('dialog');
const opener = section.querySelector('.opener');
const outside = section.querySelector('.outside');
opener.focus();
dialog.showModal();
outside.focus();
console.log('після outside.focus():',
document.activeElement.textContent.trim());
dialog.close();
console.log('після close():', document.activeElement.textContent.trim());
```

Проаналізуйте: що виведуть обидва рядки?
Потім оновіть сторінку й виконайте другий варіант: тут кнопку, з якої відкрили діалог, видалено, поки діалог відкритий.

```
const section = document.querySelector('#exp-15');
const dialog = section.querySelector('dialog');
const opener = section.querySelector('.opener');
opener.focus();
dialog.showModal();
opener.remove();
dialog.close();
```

Проаналізуйте: куди перейде фокус після `close()`? Перевірте за Live Expression.
Де це трапиться: діалог «Детальніше» і збереження нотатки, після якого картку перемальовано (Challenge 4). Поясніть, що `showModal()` робить з рештою сторінки і коли браузер сам повертає фокус.

4.5. DOM-код, написаний AI

Дослідити, що AI-агент зазвичай робить у DOM-коді, якщо його про це не просили:

- будує розмітку через `innerHTML` і шаблонний рядок із даними;
- завантажує `data/shows.json` через `fetch`, оминаючи `src/state.js`;
- додає обробник на кожну картку — один раз на старті або щоразу під час перемальовування;
- читає `event.target` без `closest()`;
- не скасовує надсилання форми;
- створює рядок статусу заново або не створює зовсім;
- вставляє опис через `innerHTML` або «очищує» його через тимчасовий `<div>`;

- робить модальне вікно з `<div>`;
- показує помилки валідації через `alert()`;
- губить фокус після перемальовування;
- додає таймери: debounce для пошуку, `setTimeout` чи `requestAnimationFrame` перед `focus()`;
- додає зайвий ARIA: `role="button"` на `<div>` без підтримки клавіатури, `aria-label`, що підміняє видимий текст (WCAG 2.5.3).

Саме це шукайте в AI-чернетках у §5.

5. Практична частина: AI-чернетка → аудит → виправлення

Starter-проєкт **«Мій список»** — каталог 30 серіалів TVmaze (та сама вибірка, що в ЛР-8). У готовому застосунку користувач фільтрує серіали за назвою й жанром, додає їх до свого списку кнопкою «До списку», бачить лічильник «У списку: N», відкриває опис кнопкою «Детальніше» і зберігає свою оцінку з нотаткою. HTML, CSS, дані й модуль стану `src/state.js` уже готові. Порожній лише JavaScript, який показує картки й реагує на дії користувача, тож відкрита сторінка показує форму й порожній список.

Ваше завдання — довести застосунок до робочого стану за чотири кроки (challenges). Кожен challenge — окрема частина застосунку, і кожен проходить той самий цикл (§5.2): AI-агент пише чернетку за коротким запитом, ви фіксуєте її в Git, перевіряєте й виправляєте власноруч. Виконуйте challenges по черзі: кожен спирається на попередні.

Як перевіряти. Точні правила для кожного challenge — у ТЗ, §4. Вони пронумеровані: С4.2 — друге правило Challenge 4. Сторінка `check.html` перевіряє правила в браузері й показує їх блоками, по одному на challenge: «С1 · Картки», «С2 · Кнопки карток» тощо. Кожне правило в блоці — під тим самим номером, що й у ТЗ: ✓ — виконано, ✗ — ні, з поясненням, що саме не так. Запускайте її скільки завгодно: вона нічого не змінює в проєкті. Те, чого `check.html` не бачить (справжня клавіатура й видимий фокус), перевіряє ручний чек-лист ТЗ, §6.

5.1. Підготовка

1. Завантажте watchlist-starter.zip і розпакуйте в **окремий репозиторій** або в теку `lab-9/` спільного репозиторію з лабораторними. Зробіть перший коміт **без змін**.
2. Відкрийте `index.html` через Live Preview у Chrome або Edge, як у §4: форма фільтрів, порожній список, Console без помилок.

3. Відкрийте так само `check.html`: «Пройдено: 0 з 33». Прочитайте, чого перевірка не бачить.
4. `check.html`, `check/`, `data/` і `src/state.js` не змінюються в жодному коміті.

5.2. Правила роботи з AI-агентом

- **Дослідження (§4).** «Мій аналіз» пишете самі, без AI-агента й до запуску.
- **Challenges.** Кожен challenge виконуйте за чотири кроки. Приклади нижче — для Challenge 2.
    1. **Чернетка AI.** Перемкніть AI-агента в IDE в режим, у якому він сам змінює файли проєкту (зазвичай — Agent), а не лише відповідає в чаті. Закрийте в редакторі вкладки з `check.html` і файлами `check/`: багато агентів додають відкриті файли до контексту. Вставте в чат **лише** запит із розділу цього challenge, слово в слово. ТЗ, `check.html` і власних уточнень агенту не давайте: треба побачити, що він зробить сам.
    2. **Коміт чернетки.** Нічого не виправляючи, закомітьте все, що змінив агент, з повідомленням `Challenge 2: AI draft`. Комітьте, навіть якщо чернетка не працює: з цим комітом викладач порівнюватиме ваші виправлення. Єдиний виняток — файли, які не змінюються в жодному коміті (§5.1). Якщо агент їх змінив, перед комітом поверніть їх командою `git restore check.html check data src/state.js`, нові файли в цих теках видаліть і запишіть це в підсумок звіту.
    3. **Аудит.** Відкрийте `check.html` і знайдіть блок цього challenge. Для кожного ✗ з'ясуйте, яке правило ТЗ порушено і чому. Перегляньте й diff чернетки на обмеження ТЗ, §5 (`innerHTML` з даними, таймери, `fetch`): `check.html` бачить не всі з них. Потім пройдіть рядки ручного чек-листа ТЗ з номером цього challenge.
    4. **Виправлення власноруч.** AI може пояснити перевірку, правило чи помилку, але **не пише** код виправлення. Коли всі рядки блоку цього challenge і блоків попередніх позначені ✓, закомітьте виправлення одним комітом: `Challenge 2: fixes`. Агент міг переписати й файли попередніх challenges, тож те, що він там зламав, теж виправляєте ви.

    Якщо чернетка з першого разу пройшла весь свій блок і не зламала попередніх, коміту виправлень немає, а в підсумку звіту вкажіть, як ви її перевірили.
    Агент часто робить більше, ніж просили: наприклад, уже в Challenge 2 будує діалог із Challenge 4. Це теж частина чернетки: залиште її, а перевіряйте й виправляйте в тому challenge, якому вона належить. Його запит однаково дайте агенту слово в слово.

Приклад запиту для пояснення (режим без редагування — ask / plan):

> Поясни, чому падає ця перевірка. Нижче — її назва й повідомлення та мій код. Не пиши виправлений код: назви механізм DOM або правило доступності, яке я, ймовірно, не врахував, і як перевірити це в DevTools.

У звіті вкажіть назву та версію агента, модель і дату роботи.

5.3. Challenge 1 — картки

Що має вийти. Після завантаження сторінки в списку «Серіали» — 30 карток, по одній на серіал з `data/shows.json`, у тому самому порядку. На картці — назва, рік, жанри, оцінка TVmaze і дві кнопки: «До списку» та «Детальніше». Над списком рядок статусу показує, скільки серіалів знайдено. Кнопки поки нічого не роблять: це Challenge 2.

Що вже є. Шаблон картки `<template id="show-card">` в `index.html` з розміткою й місцями для даних (`data-field`), порожній список `#results` і рядок статусу `#status`. Під час запуску `main.js` викликає `renderShows()` і передає їй серіали, які треба показати.

Код: функція `renderShows(list)` у `src/render.js`.

Запит для AI-агента:

> Покажи серіали з data/shows.json у списку #results: назва, рік, жанри, оцінка, кнопки «До списку» і «Детальніше».

Перевірка: відкрийте `check.html`: у блоці «C1 · Картки» всі рядки мають бути позначені ✓. Знадобляться експерименти 1, 2, 4 і 8.

5.4. Challenge 2 — кнопки карток

Що має вийти. «До списку» додає серіал до мого списку, а повторне натискання прибирає його. Кнопка показує стан (іконка заповнюється), а лічильник «У списку: N» у header оновлюється. «Детальніше» відкриває опис серіалу: сам діалог буде в Challenge 4, тут достатньо викликати `openDetails(id)`. Кнопки працюють однаково мишею й клавіатурою, а також після кожного перемальовування списку. Після натискання «До списку» фокус лишається на цій кнопці, навіть якщо картку перемальовано.

Що вже є. `toggle(id)` і `watchlist` у `src/state.js`, лічильник `#watchlist-count` у header, стилі для стану `[aria-pressed="true"]`, порожня поки функція `openDetails(id)` у `src/details.js`.

Код: функція `initList(refresh)` у `src/list.js`. `main.js` викликає її один раз, після першого показу списку; `refresh()` перемальовує список.

Запит для AI-агента:

> Зроби, щоб «До списку» додавав серіал у мій список, а «Детальніше» відкривав опис.

Перевірка: відкрийте `check.html`: у блоці «C2 · Кнопки карток» всі рядки мають бути позначені ✓. Потім пройдіть рядки C2 ручного чек-листа (ТЗ, §6). Знадобляться експерименти 3–8 і 13.

5.5. Challenge 3 — фільтри

Що має вийти. Форма над списком фільтрує картки: пошук за частиною назви спрацьовує під час введення, жанр і «Лише мій список» — одразу після вибору, і фільтри діють разом. Рядок статусу щоразу показує, скільки знайдено. Enter у полі пошуку не перезавантажує сторінку, а «Скинути» повертає всі 30 серіалів. Якщо в режимі «Лише мій список» прибрати серіал зі списку, його картка зникає, а фокус переходить на сусідню картку.

Що вже є. Форма `#filters` з полями `query`, `genre`, `mine` і кнопкою «Скинути». Функція `visibleShows({ query, genre, mine })` у `src/state.js` уже фільтрує серіали, а `main.js` передає їй те, що поверне `readFilters()`. Вам лишається прочитати значення форми й перемальовувати список після кожної зміни.

Код: функції `readFilters()` і `initFilters(refresh)` у `src/filters.js`; фокус після прибирання картки — у `src/list.js`.

Запит для AI-агента:

> Додай фільтрацію за назвою і жанром і перемикач «лише мій список».

Перевірка: відкрийте `check.html`: у блоці «C3 · Фільтри» всі рядки мають бути позначені ✓. Потім пройдіть рядки C3 ручного чек-листа (ТЗ, §6). Знадобляться експерименти 9, 12, 13 і 14.

5.6. Challenge 4 — діалог «Детальніше» і нотатка

Що має вийти. «Детальніше» відкриває модальне вікно з назвою серіалу, його описом і посиланням на сторінку TVmaze. Поки вікно відкрите, решта сторінки недоступна. Esc або «Скасувати» закривають його, і фокус повертається на кнопку, з якої вікно відкрили. Опис у даних — HTML-розмітка TVmaze, а показати його треба як звичайний текст.

У діалозі під описом — форма: «Моя оцінка» (обов'язкове ціле число від 1 до 10) і «Нотатка» (необов'язкова). Неправильної оцінки форма не приймає: під полем з'являється текст помилки, а діалог лишається відкритим. Після збереження діалог закривається, картка показує «Моя оцінка: 8/10» і нотатку, рядок статусу повідомляє про

збереження, а фокус повертається на «Детальніше» цієї картки. Коли діалог відкривають знову, форма показує збережені дані саме цього серіалу.

Що вже є. В `index.html` є `<dialog id="details">` із заголовком `#details-title`, місцем для опису `#details-summary`, посиланням `#details-link` і формою `#note-form`: поля `rating` і `note`, місце для тексту помилки `#note-rating-error`, кнопки «Зберегти» та «Скасувати». JavaScript лише заповнює й відкриває діалог. У `src/state.js` є `getShow(id)`, `saveNote()` і `notes`, а в шаблоні картки — місця для моєї оцінки й нотатки.

Код: функції `openDetails(id)` і `initDetails(refresh)` у `src/details.js`. Кнопка «Детальніше» (Challenge 2) викликає `openDetails`, а та заповнює діалог і форму. `initDetails` підключає закриття діалогу й надсилання форми. Показ оцінки на картці — у `src/render.js`.

Запит для AI-агента:

> Зроби діалог «Детальніше» з описом серіалу і формою «моя оцінка й нотатка» з валідацією.

Перевірка: відкрийте `check.html`: у блоці «C4 · Діалог „Детальніше“ і нотатка» всі рядки мають бути позначені ✓. Потім пройдіть рядки C4 ручного чек-листа (ТЗ, §6). Знадобляться експерименти 2, 8–11, 13 і 15.

5.7. Коміти

Для кожного challenge в історії Git — два коміти:

1. `Challenge N: AI draft` — те, що написав агент, без жодних змін;
2. `Challenge N: fixes` — усі ваші виправлення цієї чернетки.

Якщо чернетка одразу пройшла свій блок і не зламала попередніх, другого коміту немає (§5.2).

Якщо відкрити на GitHub коміт `Challenge N: fixes`, видно рівно те, що ви змінили в чернетці AI. На захисті ви пояснюєте за цим комітом, що саме в чернетці було не так (§7).

Після Challenge 4 пройдіть увесь ручний чек-лист ТЗ ще раз на фінальній версії й опублікуйте проєкт на GitHub Pages або Vercel (інструкція — у `README.md` starter-проєкту).

6. Вимоги до звіту

Тема листа:

```
[Веб] ЛР-9: Створення інтерактивного DOM та обробка подій
```

У листі вкажіть репозиторій і адресу опублікованого проєкту — один із двох рядків, GitHub Pages або Vercel:

```
GitHub repository: https://github.com/<username>/<repository>
GitHub Pages: https://<username>.github.io/<repository>/
Vercel: https://<project>.vercel.app/
```

Звіт повинен містити:

1. таблицю експериментів §4 (усі 15);
2. назву та версію AI-агента, модель, дату роботи;
3. результат `check.html` на опублікованій сторінці — «Пройдено: 33 з 33»;
4. ручний чек-лист ТЗ: результат обох пунктів на фінальній версії;
5. підсумок: що AI зробив правильно без підказки, а що пропускав у кожній чернетці; якщо агент змінював файли, які змінювати не можна, — які саме.

7. Усний захист

Формат здачі передбачає усну співбесіду за результатами практичної реалізації. Виправлення пишете ви, тож на захисті змінюєте код наживо, без AI, і відповідаєте на питання з демонстрацією екрана.

Орієнтовні питання:

Аудит

- Покажіть одну знахідку в AI-чернетці: рядок `check.html`, що її виявив, причину й зміну в коміті `fixes`, яка її усуває.
- Змініть код наживо: кнопка «Прибрати нотатку» на картці через наявний делегований обробник, фокус на тексті нотатки після збереження або `<select>` сортування у формі фільтрів.

Події

- Чому клік по іконці має інший `target`? Що повертає `closest()` для кліку поза кнопками?
- Чому картка, створена після запуску, працює з делегуванням, але не з обробниками на кожній картці?

Форми й фокус

- Чому Enter у полі пошуку перезавантажує сторінку, якщо `submit` ніхто не обробляє?
- Куди переходить фокус, коли елемент із фокусом видалено? Що у вашому коді цьому запобігає?

DOM і безпека

- Чому для нотатки достатньо `textContent`? Що не так із `div.innerHTML = summary; div.textContent`?

AI

- Що AI пропускав у кожній чернетці? Чому, на вашу думку?

8. Оцінювання

За роботу — **3 бали**, по одному за дослідження, практичну частину й захист, і **1 додатковий бал**.

Складник	Бали	Як оцінюється
Дослідження (§4)	1	по 0,25 за кожен підрозділ §4.1–§4.4: усі його експерименти — у таблиці з власним аналізом і фактичним результатом, а якщо вони не збіглися, — з поясненням, де аналіз був хибним (п. 1 звіту)
Практична частина (§5)	1	самоперевірка: «Пройдено: 33 з 33» у `check.html` на опублікованій сторінці — 0,5; від 28 до 32 — 0,25. Процес: коміти `AI draft` і `fixes` за §5.7, ручний чек-лист ТЗ (п. 4) і підсумок про чернетки AI (п. 5): усе є й змістовне — 0,5; бракує одного з трьох — 0,25
Захист (§7)	1	питання групи «Аудит»: знахідка в AI-чернетці й зміна коду наживо, без AI, — до 0,5; два питання з інших груп §7 — по 0,25 за повну відповідь
Додатковий бал	+1	одна з умов: термін здачі до **19.10.2026** АБО робота на парі

> **Важливо!** Без захисту за дослідження й практичну частину разом — не більше **1 бала**: захист підтверджує, що ви розумієте зроблене, а не лише отримали результат за допомогою AI. Додатковий бал зараховується й без захисту.